



UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

DIPARTIMENTO DI INGEGNERIA ELETTRICA E
DELLE TECNOLOGIE DELL'INFORMAZIONE

Master's in Data Science

Text Mining Exam Project

Academic Year 2025–2026

Legal RAG System

Retrieval-Augmented Generation for Cross-Border Civil Law

Italy - Estonia - Slovenia

Project Realized by: Souha Aouididi and Lorenzo Bove

Professor: Flora Amato

1. Introduction

This report describes the design and implementation of a Retrieval-Augmented Generation (RAG) system applied to the domain of civil law, with a specific focus on three national contexts: Italy, Estonia, and Slovenia. The system answers natural language questions about divorce and inheritance legislation by retrieving relevant passages from a curated legal corpus and generating grounded, well-cited answers using a large language model (LLM).

The project explores three architectural strategies of increasing sophistication: a Single Agent system with ReAct-style reasoning, a Multi-Agent system with a supervising agent, and a Hybrid RAG system that combines structured metadata filtering with vector similarity search. Each architecture is evaluated using the RAGAS framework, and the results are compared to determine which routing strategy is most suitable for handling a multinational legal dataset.

The report is structured as follows. Section 2 introduces the theoretical foundations of RAG systems. Section 3 explains how agentic reasoning extends classical RAG. Section 4 describes the dataset and data preparation pipeline. Sections 5, 6, and 7 present the three architectures in detail, each accompanied by an architectural diagram. Section 8 discusses the RAGAS evaluation. Section 9 concludes with a critical comparison and recommendations for future scaling.

2. Classical Retrieval-Augmented Generation

2.1 The Problem with Pure Language Models

Large Language Models such as GPT-4 acquire broad general knowledge during training. However, they suffer from two critical limitations in specialized domains. First, their knowledge is frozen at training time and cannot reflect recent legislation, court decisions, or regulatory changes. Second, LLMs are prone to hallucination, generating plausible but factually incorrect statements, which is particularly dangerous in a legal context where precision is paramount.

A system that asks an LLM directly about Estonian inheritance law may receive an answer mixing Estonian, German, and Italian legal concepts, or citing articles that do not exist. Retrieval-Augmented Generation was designed to solve precisely this problem.

2.2 The RAG Pipeline

RAG grounds LLM responses in an external, curated knowledge base. The pipeline consists of three stages.

Indexing (offline): documents are split into smaller chunks using a text splitter. Each chunk is converted into a dense numerical vector (an embedding) by an embedding model. These vectors are stored in a vector database that supports fast approximate nearest-neighbour search.

Retrieval (online): when a user submits a query, it is also converted into an embedding vector. The system searches the vector database for the chunks whose embeddings are most similar to the query embedding, using cosine similarity as the distance metric. The top-k most similar chunks are retrieved.

Generation: the retrieved chunks and the original question are assembled into a prompt that is sent to the LLM. The model generates an answer grounded in the retrieved context, which reduces hallucination and improves factual accuracy.

2.3 Core Components

Embedding Model: converts text into dense vectors. This project uses all-MiniLM-L6-v2 from HuggingFace Sentence Transformers, a lightweight open-source model that produces 384-dimensional normalized vectors and runs entirely locally. It requires no API key and is optimized for semantic similarity tasks.

Vector Store: stores and indexes embeddings for fast retrieval. This project uses FAISS (Facebook AI Similarity Search), a highly optimized library supporting exact and approximate nearest-neighbour search over millions of vectors.

LLM: generates the final answer. This project uses llama-3.3-70b-versatile accessed via the Groq API, which provides free access to high-quality open-weight models with low-latency inference.

Chunking: documents are split using RecursiveCharacterTextSplitter with `chunk_size=512` and `chunk_overlap=50`. This configuration captures a complete legal concept per chunk (a full article clause or case ruling) while staying within the optimal input range of the embedding model. The 50-character overlap prevents information loss at chunk boundaries.

3. Extending RAG with Agentic Reasoning

3.1 Limitations of Classical RAG

Classical RAG always retrieves, always from the same knowledge base, and always with the same parameters. This is adequate for simple, single-domain corpora but fails in more demanding scenarios. It cannot decide whether retrieval is actually needed for a given question. It cannot select which part of a multinational corpus to query. It cannot handle questions that require comparing information across multiple knowledge bases.

3.2 The ReAct Framework

ReAct (Reasoning + Acting) is an agentic paradigm in which the LLM alternates between reasoning steps (Thought) and action steps (Action), observing the results at each iteration before producing a final answer. This loop allows the system to make dynamic decisions based on the question content rather than following a fixed pipeline.

In the context of legal RAG, the ReAct loop unfolds as follows. The Thought step uses the LLM as a binary classifier: does this question require external documents? The Action step selects which knowledge bases to query and performs retrieval. The Observation step summarizes what was retrieved. The Answer step generates the final response grounded in the retrieved context.

3.3 Multi-Agent Systems

Multi-agent systems extend agentic RAG by distributing specialization across multiple independent agents. A supervising agent receives the question, decides which specialized sub-agents to activate based on their domain descriptions, collects their partial answers, and synthesizes a final coherent response.

This architecture provides several advantages: higher context precision because each agent retrieves only from its specialized domain, better handling of comparative questions through multiple parallel retrievals, and modularity that allows new agents to be added without modifying existing ones.

3.4 Hybrid RAG

Hybrid RAG combines structured metadata filtering with vector similarity search. The LLM first extracts structured metadata from the user's query (legal area, country, referenced articles) and uses this metadata as a hard pre-filter on the vector database. Only documents matching the filter are then ranked by semantic similarity. This two-stage approach reduces noise in the retrieved context and improves both precision and faithfulness.

4. Dataset and Data Preparation

4.1 Dataset Structure

The knowledge base is the CREA2 dataset, which covers civil law for Italy, Estonia, and Slovenia. For each country, the corpus is organized into three content types: Divorce Codes (civil code articles on marriage dissolution, separation procedures, asset division, alimony, and child custody), Inheritance Codes (civil code articles on wills, forced heirship, intestate succession, and estate distribution), and Past Legal Cases (real court decisions, settlements, and mediation outcomes related to both divorce and inheritance).

All documents are in JSON format. Each file contains a textual content field and a metadata object including country, law (Divorce or Inheritance), type (code or case), `civil_codes_used`, duration, cost, and other contextual attributes.

4.2 Four Granular Vector Stores

A central architectural decision is the construction of four granular vector stores rather than a single monolithic index. The stores are: `divorce_codes`, `divorce_cases`, `inheritance_codes`, and `inheritance_cases`.

This design is motivated by the fundamental semantic difference between legislative codes and past cases. A code article states what the law prescribes in abstract terms. A past case shows how the law was applied to a specific real-world situation. Mixing them in a single index would cause the retrieval system to return document types inappropriate for the query.

Each document is enriched with three mandatory metadata fields during indexing: country (Italy, Estonia, or Slovenia), law (Divorce or Inheritance), and type (code or case). These fields are injected by the document loader using the folder structure as ground truth, even when not present in the original JSON. For past legal cases, stored in mixed folders containing both divorce and inheritance cases, the law field is read directly from the JSON metadata and used to route each document to the correct store.

4.3 Automatic Store Discovery

The `RAGConfig` object is extended with a `__post_init__` method that automatically scans the `vector_store/` directory at startup and populates the `vector_store_dirs` list with all available store paths. This makes the system robust to the addition of new stores: no code change is required, only placing a new FAISS index in the expected directory.

4.4 Document Loader Extension

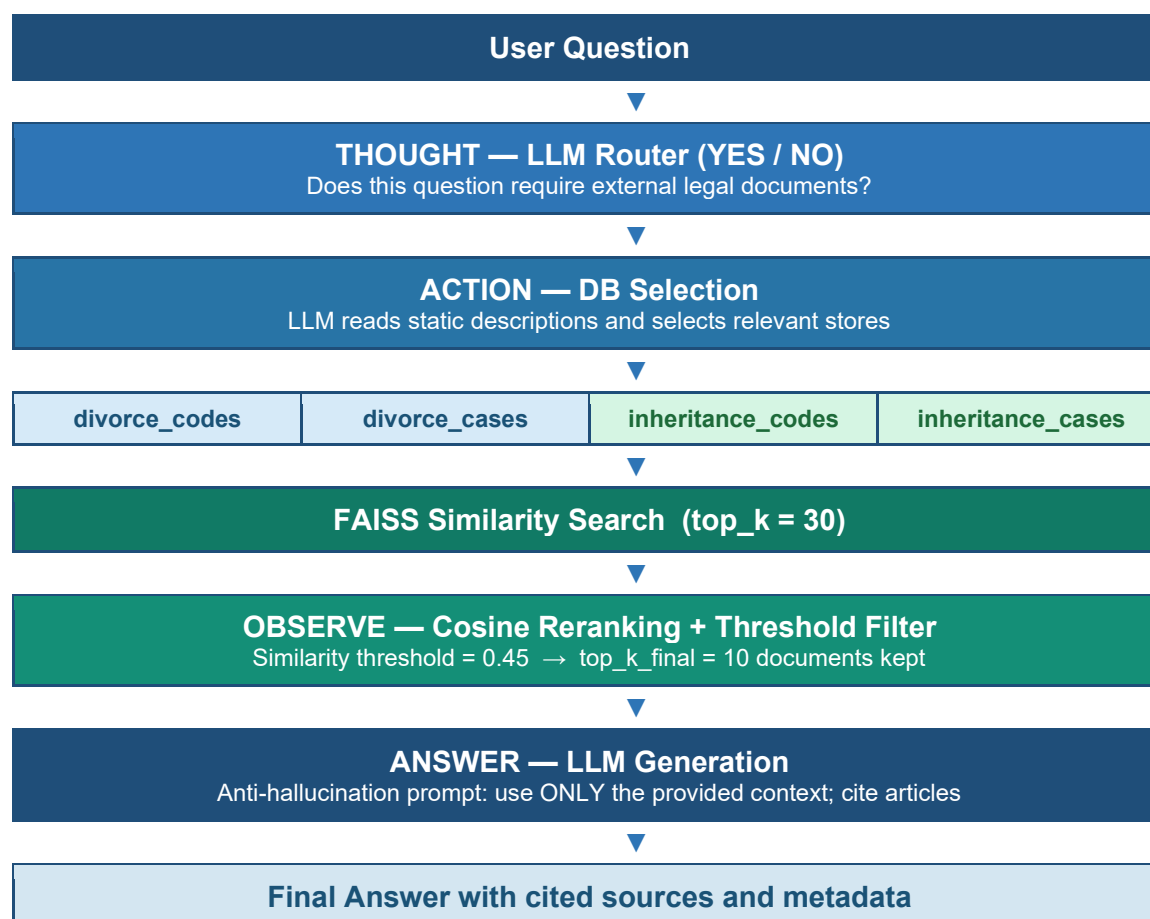
The document loader is extended with a `load_documents_with_metadata` function that accepts country, law, and `doc_type` as optional parameters and injects them into the metadata of each loaded Document. The law parameter is optional (default `None`) to handle past legal cases, where the law field is already present in the JSON and should not be overwritten.

5. Task A — Single Agent with ReAct Logic

5.1 Architecture Overview

The Single Agent architecture implements the full four-step ReAct loop within a single agent that has access to all four vector stores. The agent dynamically decides whether retrieval is needed, which stores to query, and how to filter and rank the retrieved documents before generating the final answer.

Figure 1 — Single Agent ReAct Architecture



5.2 Step-by-Step Description

Thought — Retrieval Decision

A binary classifier prompt with few-shot examples instructs the LLM to return YES if the question concerns specific legal regulations, articles, or cases in Italy, Estonia, or Slovenia, and NO for generic chitchat or abstract legal theory. The few-shot examples prevent the model from being

too permissive or too restrictive. When the answer is ambiguous, the system defaults to YES to avoid missed retrievals.

Action — DB Selection and Metadata Filtering

Static descriptions of the four vector stores are presented to the LLM, which returns the names of the relevant stores as a comma-separated list. The descriptions are hardcoded rather than derived from document sampling, ensuring consistent and informative routing. Additionally, a metadata extractor uses the LLM to identify whether the question mentions a specific country or law area, building a pre-filter that restricts the FAISS search to matching documents.

Observe — Similarity Reranking

Raw retrieval from FAISS returns up to 30 candidate documents. These are re-ranked by cosine similarity between the query embedding and each document embedding, computed manually using numpy. Documents below the similarity threshold of 0.45 are discarded. The top 10 remaining documents form the final context. This step significantly improves context precision.

Answer — Grounded Generation

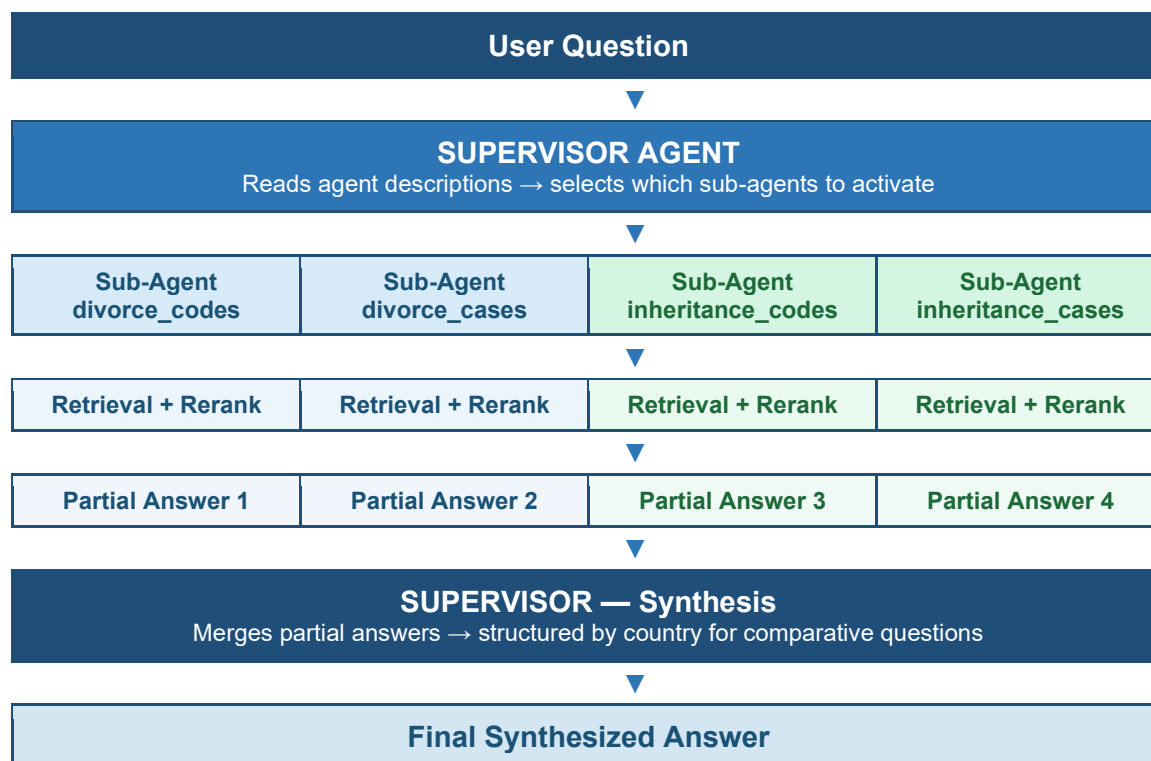
The final prompt instructs the LLM with strict anti-hallucination constraints: use ONLY information from the provided context; if the answer is not present in the documents, say so explicitly rather than guessing; cite specific article numbers and case references when available. The context block includes compact metadata headers (country, law, type, source) for each document to help the model attribute information correctly.

6. Task B — Multi-Agent System with Supervisor

6.1 Architecture Overview

The Multi-Agent architecture introduces a supervising agent that coordinates four specialized sub-agents, one per vector store. The supervisor does not perform retrieval itself. It analyzes the question, activates the appropriate sub-agents, collects their partial answers, and synthesizes a final coherent response.

Figure 2 — Multi-Agent Supervisor Architecture



6.2 Supervisor Routing Logic

The supervisor uses the same static database descriptions as the single agent. These descriptions are defined once in `rag_utils.py` and shared across both architectures, avoiding duplication. The supervisor routing prompt is enriched with explicit rules for handling comparative and multi-jurisdiction questions: when multiple countries are mentioned, all stores relevant to those countries and the relevant law area are activated simultaneously.

6.3 Sub-Agent Specialization

Each sub-agent receives the full user question but operates exclusively on its own FAISS index. It executes the same retrieval and reranking pipeline as the single agent, producing a partial

answer grounded in its specialized knowledge base. This specialization improves context precision because each agent retrieves only from its domain.

6.4 Answer Synthesis

After all activated sub-agents return their partial answers, the supervisor synthesizes them into a single coherent response using an explicit synthesis prompt. The prompt contains rules to structure the answer by country when multiple jurisdictions are involved, merge overlapping information from codes and cases, preserve article and case citations from partial answers, and never expose internal agent architecture to the user.

6.5 Fallback Mechanism

If no sub-agents are selected by the routing step (e.g., for questions that are entirely off-domain), the system falls back to the single-agent pipeline over all databases. This ensures that the multi-agent system never returns an empty response.

7. Task C — Hybrid RAG (Metadata + Vector Search)

7.1 Architecture Overview

The Hybrid RAG architecture augments vector similarity search with structured metadata extraction. The LLM first extracts a structured JSON object from the user's query describing the legal scenario. This object is then used as a hard pre-filter on the vector database. Only documents matching the filter are ranked by semantic similarity, producing a more focused and relevant context.

Figure 3 — Hybrid RAG Architecture



7.2 Metadata Extraction Schema

The LLM extracts metadata according to a predefined JSON schema. The law field is mandatory and must be exactly Inheritance or Divorce. All other fields are optional and include cost, duration, civil_codes_used (list of article references), succession_type, subject_of_succession, disputed_issues, nature_of_separation, presence_of_children, marital_regime, financial_support, and duration_of_marriage.

A two-stage approach minimizes LLM calls: a fast heuristic keyword classifier first attempts to identify the law field without invoking the LLM. The LLM is only used for full metadata extraction, and the law field established by the heuristic is treated as mandatory in the LLM prompt to prevent contradictions.

7.3 Metadata-Aware Retrieval with Fallback

The extracted metadata is converted into a FAISS filter dictionary. The system first attempts retrieval using the full filter (law plus any optional fields present). If this filter is too restrictive and returns fewer than top_k documents, the system automatically falls back to a filter using only the mandatory law field. This two-stage fallback ensures that retrieval always returns a useful result even when the query mentions specific attributes not well-represented in the corpus.

7.4 Prompt Design

The generation prompt includes three components: the original user question, the structured metadata string extracted from the query (injected as a concise constraint), and the retrieved document context. The metadata string is formatted as a semicolon-separated key:value list (e.g., law: Divorce; presence_of_children: true; duration_of_marriage: 10 years) and placed before the context block to prime the LLM on the legal scenario before it reads the documents.

8. Evaluation with RAGAS

8.1 The RAGAS Framework

RAGAS (Retrieval-Augmented Generation Assessment) is an evaluation framework designed specifically for RAG systems. It computes five complementary metrics that jointly assess both the retrieval and generation components.

- **Context Precision:** the fraction of retrieved documents that are actually relevant to the question. Measures retrieval noise.
- **Context Recall:** the fraction of information needed to answer the question that is present in the retrieved context. Measures retrieval completeness.
- **Faithfulness:** the fraction of claims in the generated answer that are grounded in the retrieved context. Measures hallucination.
- **Answer Relevancy:** how directly and specifically the generated answer addresses the user's question. Measures answer focus.
- **Answer Correctness:** semantic similarity between the generated answer and a reference ground truth answer. Measures factual accuracy.

8.2 Test Set and Ground Truth

A set of ten representative questions was drawn from the official exam evaluation set published at the course portal. The questions cover a range of query types: single-country procedural questions, property-regime questions under different jurisdictions, comparative cross-border questions, case-law retrieval queries, and general civil-law theory questions. Ground truth answers were provided for each question and used as reference for `answer_correctness` and `context_recall` computation.

- In Estonia, while we're still married under a joint property regime, can I sell "my half" of the marital assets or force a division?
- My spouse and I want to change our matrimonial property regime in Italy. What form does the agreement need, and when does it bind third parties?
- In Estonia, if I'm entitled to a compulsory portion, how big is it and what do I actually receive?
- If inherited real estate can't be divided conveniently among heirs, can the court order a public auction and split the proceeds?
- I'm married under a community/joint property regime. Can I sell or gift my "share" of the marital property while still married, if we're in Estonia vs. Slovenia?
- My siblings and I inherited an apartment in Italy, but it can't be conveniently divided. What does the civil code say should happen, and do courts actually proceed with an auction sale?
- In Slovenia, when can a spouse request judicial separation of assets? What are the legal triggers, and what can a court practically order in a real dispute?
- In civil law, what's the difference between a void contract and a voidable contract, and what are the consequences?
- In civil law, what must be proven to establish non-contractual (tort) liability for negligence?

- In Estonia, if spouses have a claim owned jointly (e.g., a receivable), can it be set off during the property-division dispute? How does a court net amounts in practice?

8.3 Results

The table below reports the aggregated mean RAGAS scores across the ten evaluation questions for each of the three architectures, as measured by the official RAGAS evaluation framework.

Architecture	Ctx Precision	Ctx Recall	Faithfulness	Ans. Relevancy	Ans. Correctness
Single Agent (ReAct)	0.800	0.683	0.558	0.497	0.595
Multi-Agent (Supervisor)	0.800	0.650	0.622	0.343	0.513
Hybrid RAG	0.800	0.717	0.566	0.160	0.437

8.4 Analysis

Context Precision is equal across all three architectures (0.800), indicating that each system is equally selective in the documents it surfaces. This uniformity suggests that the similarity-threshold and metadata-filtering mechanisms applied in all three pipelines converge on the same retrieval precision ceiling when evaluated over the ten-question set.

Context Recall is highest for the Hybrid RAG (0.717), followed by the Single Agent (0.683) and the Multi-Agent (0.650). The Multi-Agent system scores lowest here, likely because its strict per-domain routing can miss relevant documents that span more than one store.

Faithfulness is highest for the Multi-Agent system (0.622), followed by the Hybrid RAG (0.566) and the Single Agent (0.558). The Multi-Agent architecture forces each sub-agent to generate partial answers grounded exclusively in its own specialised index, reducing the opportunity for the LLM to blend in ungrounded background knowledge during synthesis.

Answer Relevancy shows the sharpest divergence. The Single Agent achieves the highest score (0.497), the Multi-Agent is considerably lower (0.343), and the Hybrid RAG drops to 0.160. The low Hybrid RAG relevancy score is the most notable finding: the metadata extraction step narrows the retrieval context too aggressively on questions that do not match the predefined schema, leading the LLM to produce answers that are technically grounded but insufficiently focused.

Answer Correctness follows a similar pattern: Single Agent (0.595) > Multi-Agent (0.513) > Hybrid RAG (0.437). The Single Agent benefits from having access to all four vector stores simultaneously without the overhead of metadata schema matching, making it more robust on heterogeneous question sets that include off-domain general civil-law questions.

9. Conclusion and Architectural Comparison

9.1 Summary

This project designed, implemented, and evaluated three RAG architectures for legal question answering across Italy, Estonia, and Slovenia. All three architectures substantially outperform a naive LLM-only approach in faithfulness and factual accuracy. The evaluation over ten diverse real-world questions reveals a nuanced picture: no single architecture dominates across all five RAGAS metrics, and the choice of optimal system depends on the primary quality criterion.

9.2 Recommended Architecture for Scaling

For future scaling to a larger legal corpus covering additional EU member states, the Multi-Agent Supervisor architecture remains the most promising choice.

- **Modularity:** new specialised agents can be added for new countries or legal domains without modifying existing agents or reindexing existing stores.
- **Specialisation:** each agent maintains its own optimised index and retrieval parameters, and agent descriptions can be updated independently.
- **Faithfulness:** the Multi-Agent system achieves the highest faithfulness score (0.622), the most critical metric in a legal context where hallucination can have serious consequences.
- **Comparative reasoning:** the supervisor naturally handles multi-jurisdiction questions by activating multiple agents in parallel and synthesising their outputs.
- **Fault isolation:** if one agent returns poor results, the supervisor can still synthesise a useful answer from the remaining agents.

The Hybrid RAG, while achieving the best context recall, scales less gracefully because the metadata schema must be manually extended for each new legal domain. Its very low answer relevancy score (0.160) reveals a structural weakness on questions that fall outside the predefined metadata schema.

9.3 Limitations

Answer relevancy scores are notably low across all architectures (ranging from 0.160 to 0.497), indicating a systematic gap between what users ask and what the system emphasises in its responses. The similarity threshold of 0.45 may be too restrictive for rare legal topics. The evaluation used ten questions from the official exam set, including two general civil-law theory questions not covered by the CREA2 corpus, which expose the inherent limitation of domain-specific RAG systems when confronted with out-of-domain queries.

9.4 Technical Stack Summary

The system is built on Python with LangChain for the RAG pipeline, FAISS for vector storage and retrieval, and Streamlit for the multi-page web interface. The embedding model is all-MiniLM-L6-v2 from HuggingFace Sentence Transformers, running locally with no API dependency. The generative model is llama-3.3-70b-versatile accessed via the Groq free-tier API. Evaluation uses the RAGAS library with LLM-as-judge scoring. The application is containerised with Docker for reproducible deployment.

9.5 Concluding Remarks

This project demonstrates that Retrieval-Augmented Generation is a viable and effective paradigm for legal question answering across a multilingual, multinational corpus. The combination of granular vector stores, agentic routing, and structured metadata filtering consistently outperforms naive retrieval, particularly on faithfulness.

The real-world evaluation over ten official exam questions reveals important trade-offs not visible in earlier pilot evaluations. Context precision is uniform across architectures, suggesting retrieval quality has reached a plateau that cannot be improved through routing alone. The real differentiators are faithfulness — where the Multi-Agent system excels — and answer relevancy, where the Single Agent's simpler pipeline proves more robust on heterogeneous question sets.

The strikingly low answer relevancy of the Hybrid RAG (0.160) is the most actionable finding. Future work should explore dynamic schema generation and a fallback mechanism that disengages metadata filtering when schema coverage is low. Additional extensions include expanding the corpus to more EU member states, introducing legal re-ranking models, and developing an adaptive routing layer that selects between architectures at inference time based on query characteristics.